



Lenguaje de programación

M1. Fundamentos de programación



Lenguaje de programación

Un lenguaje de programación es un sistema formal, compuesto por reglas, sintaxis y semántica, el cual permite a los desarrolladores comunicar instrucciones a una computadora, para que realice tareas específicas. Es el puente entre los humanos y las máquinas, traduciendo las ideas en acciones que un dispositivo electrónico puede ejecutar.

Características principales de un lenguaje de programación

Para que algo se considere un lenguaje de programación, debe cumplir con ciertas características:

- **Sintaxis estructurada:** Define un conjunto de reglas que especifican cómo deben escribirse las instrucciones, por ejemplo, el uso de llaves { } en lenguaje C o las sangrías en Python.
- **Semántica clara:** Establece el significado de las instrucciones escritas, por ejemplo, la declaración `int x = 10;` en lenguaje C, indica que x es una variable entera inicializada con el valor 10.
- **Abstracción:** Oculta la complejidad del hardware, permitiendo a los programadores enfocarse en el problema a resolver y no en los detalles técnicos del dispositivo.
- **Turing completo:** Un lenguaje de programación debe ser capaz de expresar cualquier cálculo que una máquina de Turing pueda realizar con el tiempo y los recursos suficientes.

- **Portabilidad:** Los programas escritos en un lenguaje de alto nivel pueden ejecutarse en diferentes sistemas, con mínimas modificaciones (siempre que haya un compilador o intérprete adecuado).
- **Capacidad de interacción:** Permite interactuar, ya sea con el hardware, con otros programas y con sistemas externos, mediante interfaces de programación (APIs).

Variables y operaciones

Una variable representa un contenedor o un espacio en la memoria física o virtual de una computadora, donde se almacenan distintos tipos de datos (**valores**), durante la ejecución de un programa. A cada variable se le asigna un nombre descriptivo o un **identificador** que se refiere al valor guardado. Los datos almacenados pueden cambiar de valor o ser constantes y pueden ser usados en cálculos, operaciones como datos de entrada y salida, comparaciones y otros tipos de procesamiento de datos. Por ejemplo, en un programa de contabilidad, una variable puede almacenar el saldo de una cuenta, y otra variable representa el monto de una transacción. A través de las variables, puedes realizar fácilmente operaciones matemáticas para actualizar el saldo de la cuenta.

Una variable en programación se caracteriza por:

- El nombre único que la identifica dentro del programa.
- El valor guardado, que puede cambiar durante la ejecución del programa.

- El tipo de datos que se almacena, como números, textos, lógicos, etc.
- El alcance, que puede ser global, si se cuenta con la opción para acceder desde cualquier parte del programa, o local, cuando solo existe la posibilidad de acceder dentro de un bloque de código.

Los tipos de variables dependen del lenguaje de programación. Cada lenguaje tiene su propio tipo de datos y formas de definir variables. El tipado puede ser estático; cuando los tipos de variables se establecen a la hora que se declaren, o dinámico, cuando el tipo se determina y se asigna durante la ejecución del programa.

A continuación, veamos algunos ejemplos del uso de variables en programación:

En Java

```
string nombre = "Juan"; // se declara una variable de tipo cadena de
caracteres llamada nombre y se le asigna un valor
int edad = 25; // se declara una variable de tipo numérico entero
llamada edad y se le asigna un valor
float puntosGanados = 7.35; // se declara una variable de tipo flotante
llamada puntosGanados y se le asigna un valor
```

En PHP

```
$nombre = "María";
$edad = 31;
$puntos_ganados = 6.01;
```

En JavaScript

```
let nombre = "Pedro";  
let edad = 28;  
let puntosGanados = 8.26;
```

En Python

```
nombre = "Carmen";  
edad = 33;  
puntos_ganados = 9.99;
```

Tipos de datos en programación

Un tipo de datos en programación, es una clasificación que define qué tipo de valores puede almacenar una variable, y qué operaciones pueden realizarse con esos valores. Es un concepto fundamental que ayuda a organizar, manipular y utilizar los datos de manera eficiente en un programa. La razón para utilizarlos, es el ayudar a los programadores a definir el propósito de una variable; optimizar el uso de la memoria; garantizar validez en operaciones, y facilitar la comprensión y el control de los programas.

Los principales tipos de datos son:

- **Tipos primitivos**
 - o Numéricos
 - Enteros. Representan números sin decimales.

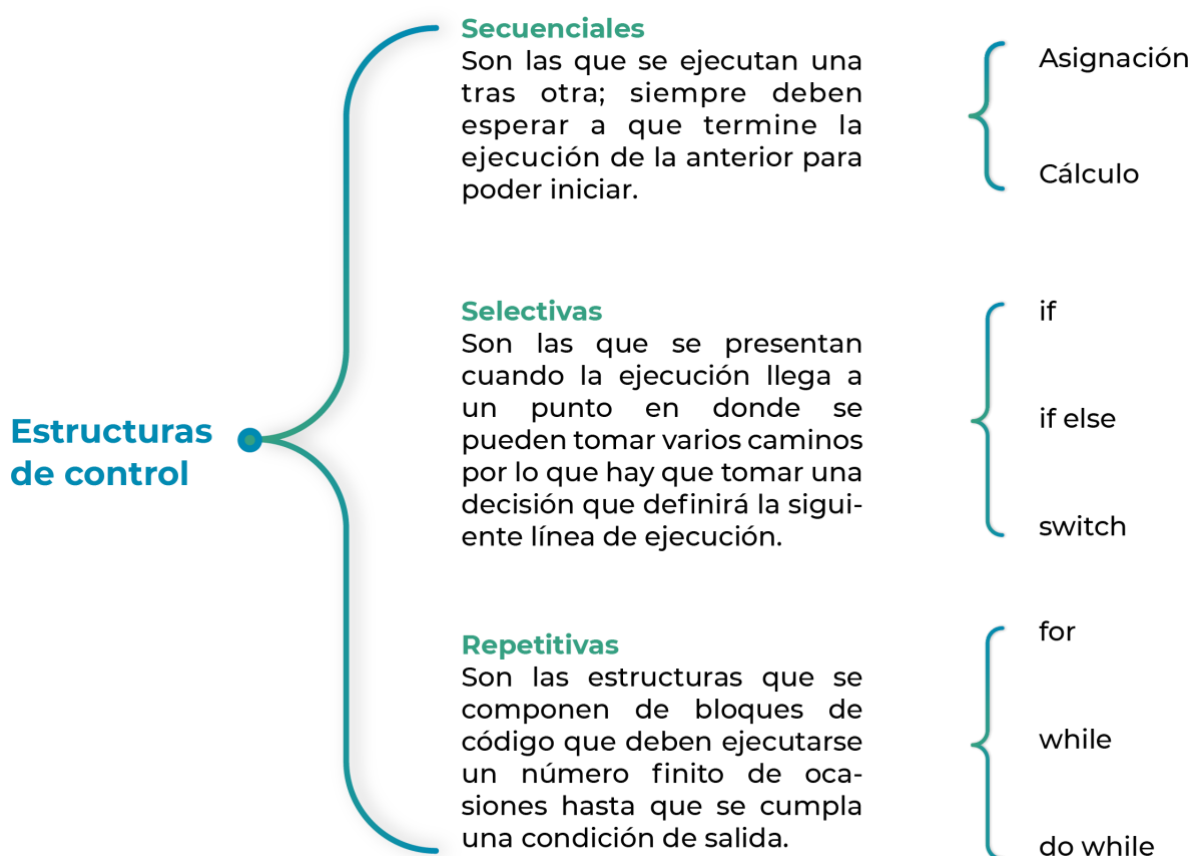
- Flotantes. Representan números con decimales.
- **Texto**
 - Caracteres. Representan un solo carácter.
 - Cadenas. Representan texto compuesto por múltiples caracteres.
- **Booleanos**
 - Representan valores lógicos como true o false.
- **Tipos compuestos**
 - **Arreglos.** Conjuntos ordenados de datos homogéneos, es decir, del mismo tipo.
 - **Estructuras.** Agregan múltiples valores de diferentes tipos.
- **Tipos abstractos**
 - **Listas, pilas, colas y conjuntos.** Son colecciones de valores más avanzadas que dependen de la lógica de programación.
 - **Mapas o diccionarios.** Pares de clave-valor, para asociar datos.

Estructuras de control

Las estructuras de control, también conocidas como instrucciones de control, sirven para definir el flujo de ejecución de un programa. En lugar de ejecutar las líneas de código de manera secuencial, las estructuras de control determinan qué partes del código se ejecutan, cuántas veces, y en qué condiciones.

Al momento de programar, definimos opciones de ejecución, según se presente una situación u otra, de esta manera podremos predeterminar qué instrucción o instrucciones se ejecutarán, y en qué orden lo harán o cuántas veces se ejecutará determinada instrucción.

Existen tres tipos:



Secuenciales

- **Asignación:** Instrucción que almacena un valor en una variable.
- **Cálculo:** Operación aritmética o lógica que produce un resultado.
- **if:** Evalúa una condición y ejecuta un bloque de código si es verdadera.

Selectivas

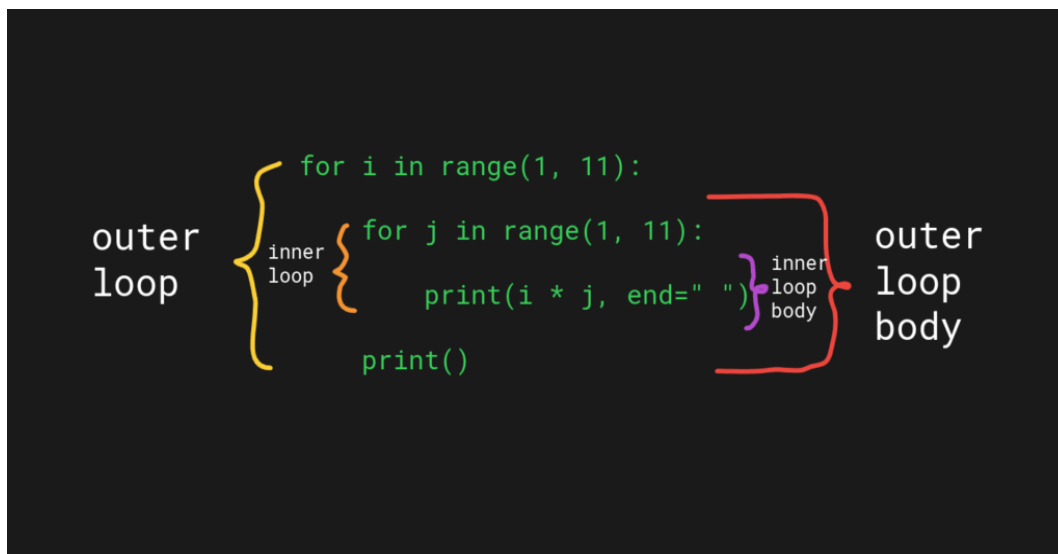
- **if else:** Evalúa una condición y ejecuta un bloque si es verdadera. De lo contrario, ejecuta otro bloque.
- **switch:** Selecciona, entre múltiples bloques de código, según el valor de una expresión.

Repetitivas

- **for:** Ejecuta un bloque de código durante un número definido de veces, utilizando un contador.
- **while:** Ejecuta un bloque de código mientras una condición sea verdadera.
- **do while:** Ejecuta un bloque de código, al menos una vez, y luego continúa con la ejecución, mientras la condición sea verdadera.

Anidamiento

El anidamiento, en el ámbito de la programación, se refiere a la práctica de incluir una estructura de control dentro de otra. Esto permite manejar flujos de control más complejos, donde las decisiones, repeticiones o combinaciones de ambas, dependen de condiciones múltiples o de pasos intermedios dentro de un programa.



Tipos de anidamiento:

- **Anidamiento de estructuras condicionales:** Se usan cuando una decisión depende de otra.
- **Anidamiento de bucles:** Un bucle se ejecuta dentro de otro, generalmente para trabajar con estructuras más complejas, como matrices o tablas.
- **Anidamiento mixto:** Combina bucles y estructuras condicionales, lo que resulta muy útil para flujos de control más detallados.

Las principales ventajas de utilizar el anidamiento son:

- **Flexibilidad:** Permite manejar problemas con lógica compleja.
- **Modularidad:** Facilita dividir problemas grandes en pasos más pequeños y manejables.
- **Control detallado:** Ofrece un nivel de precisión elevado, al gestionar condiciones o repeticiones.

Para utilizar el anidamiento de manera efectiva, debes de tener en cuenta las mejores prácticas, como: limitar el nivel de anidamiento en un máximo de 3 niveles, pues puede dificultar su comprensión; refactorizar el código para un fin específico en funciones, mejorando la lectura y documentar cada bloque para explicar su propósito.

Funciones

En el contexto de la programación de software, una función es un bloque de código que realiza una tarea específica, tiene un nombre identificable y puede ser invocado (llamado) desde otras partes del programa. Las funciones están diseñadas para dividir un programa en partes más pequeñas, reutilizables y manejables, facilitando la organización y el mantenimiento del código.

Aunque la sintaxis varía entre lenguajes, la estructura de una función típicamente incluye un **nombre o identificador único** para invocarla, y el **cuerpo**, que contiene el conjunto de instrucciones que realiza la tarea. Las funciones pueden solicitar **parámetros** que son valores de entrada, estos se especifican dentro de paréntesis y pueden arrojar un **valor de retorno** que es el resultado de la ejecución de la función.

Ventajas de usar funciones

- **Modularidad:** Dividen un programa en bloques independientes y manejables.
- **Reutilización:** Evitan la duplicación de código, reduciendo errores y esfuerzo.

- **Claridad:** Mejoran la legibilidad y organización del código.
- **Mantenimiento:** Facilitan la actualización de programas, ya que los cambios en la lógica se realizan solo en la función correspondiente.
- **Pruebas:** Hacen que el código sea más fácil de probar, ya que las funciones pueden evaluarse de forma independiente.

<pre># Conversión de metros a kilómetros metros1 = 1500 kilometros1 = metros1 / 1000 print(f"{metros1} metros son {kilometros1} kilómetros") # Conversión de metros a kilómetros metros2 = 3000 kilometros2 = metros2 / 1000 print(f"{metros2} metros son {kilometros2} kilómetros") # Conversión de metros a kilómetros metros3 = 500 kilometros3 = metros3 / 1000 print(f"{metros3} metros son {kilometros3} kilómetros")</pre>	Sin funciones El código es repetitivo y debes leer cada línea para conocer qué se está ejecutando. Si deseas cambiar la fórmula debes hacerlo muchas veces.
<pre># Definición de la función para la conversión def convertir_metros_a_kilometros(metros): return metros / 1000 # Uso de la función valores = [1500, 3000, 500] # Lista de valores en metros for metros in valores: kilometros = convertir_metros_a_kilometros(metros) print(f"{metros} metros son {kilometros} kilómetros")</pre>	Con - funciones • El código es relativamente más sencillo de interpretar pues al leer el propósito de la función una sola vez, sabes que siempre se ejecuta lo mismo. • Si deseas cambiar algún aspecto del cálculo, se realiza sólo una vez, asegurando la consistencia de los cálculos.

Usar funciones no solo reduce la duplicación de código, sino que también mejora la claridad, facilita el mantenimiento y permite construir programas escalables.

Referencias:

- Alcolea, A. (2023). *La historia de los lenguajes de programación*. Computer Hoy. Recuperado el 06 de Diciembre de 2024, de <https://computerhoy.20minutos.es/reportajes/tecnologia/historia-lenguajes-programacion-428041>
- Sandoval, E. (2023). *¿Qué es una variable en programación?* Escuela Británica de Artes Creativas y Tecnología. Recuperado el 06 de Diciembre de 2024, de <https://ebac.mx/blog/variable-en-programacion>
- Hernández, S., & Daluz, D. (2022). *Estructuras de control*. REA Ceibal. Recuperado el 06 de Diciembre de 2024, de <https://rea.ceibal.edu.uy/elp/estructuras-de-control-en-programacion/index.html>
- Jalli, A. (2024). *Nested loops in Python: A complete guide*. Codingem. Recuperado el 06 de Diciembre de 2024, de <https://www.codingem.com/nested-loops-in-python/>

Elaboró

Contenido: Mtro. Marco Antonio Ruiz Valdez